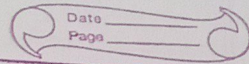


const is Fixed, if we initialise it with any value, it is fixed, it would not change.



Ex → `<Script>`
 `console.log("Hello");`
 `</Script>`

* Declaring variables in JS

1. In JS, we have 4 ways to declare a variable and they are:

- (i) `var a = 10;`
- (ii) `let a = 10;`
- (iii) `const a = 10;`
- (iv) `a = 10;`

2. `var`, `let` and `const` are keyword.

3. `let` and `const` are added to JS from ES6 onwards while `var` is available from original version of JS.

4. `var`, `let` and `const` have very imp't differences but we can discuss them only when we have knowledge of the topic called Scope which is a part of chapter function.

* Rules for declaring variable

Rule 1 → variable name can contain alphabet as well as digits but it must begin with alphabet.

EX $\rightarrow$

$\checkmark$	let $a_1?$
$\checkmark$	let $a b?$
$\checkmark$	let $a?$
$\times$	let $1?$
$\times$	let $12?$
$\checkmark$	let $a 12?$

Rule 2 → variable name can contain special symbol also but JS allow only 2 types of special symbols which are \$ and -

variable name can even begin with these symbols-

Ex → X let roll no ;
 ↑
 space not
 allowed

- ✓ let roll-no;
- ✓ let roll \$no;
- ✓ let rollno;
- ✓ let roll#no;
- ✓ let \$1;
- ✓ let -1;

Rule 3 → Unlike HTML & CSS, variable names in JavaScript are case sensitive.

let a = 10;
console.log(A); X not defined

This is an error message

Rule 4 → In JavaScript, the default value of any variable is undefined.

C	Java	JS
int a; printf("%d", a);	int a; SOP(a);	let a; console. log(a);
	error: a might not have been initialised	(undefined) not error but a special message

* special note

→ undefined is only for variables declared with var and let

→ If a variable is declared

Date _____
Page _____

with `const` and left uninitialized then JS will give error.

For ex - the foll. code will not run

```
const a;  
console.log(a);
```

* Data types

→ Since JS is also a lang. so like another prog. language it also has concepts of data types.

→ although we never mention data type name while creating variables in JS because JS automatically decides a variables data type but we must have knowledge of all data types in JS so that we can better understand behaviour of our code.

→ In JS datatypes are broadly divided in 2 categories - ✓

Date _____
Page _____

(a) Primitive types (also called value types)

(b) non-Primitive types (also called reference types)

→ In primitive types JS has 7 data types present

- | | |
|---------------|-------------|
| (a) Boolean | (b) Numbers |
| (c) String | (d) Null |
| (e) undefined | (f) Symbol |
| (g) BigInt | |

→ In non-Primitive data types technically we have only 1 type called object but since we have diff types of object in JS so we can say they are of 3 types-

- | | |
|--------------|-----------|
| (a) object | (b) Array |
| (c) Function | |

1. Boolean

→ In JS, booleans are logical values represented as true and false.

→ JS always uses booleans to

represent result of a condition.

EX → `let a = 10 > 5;`
`console.log(a);`
`console.log(typeof a);`
`a = 6 < 4;`
`console.log(a);`
`console.log(typeof a);`

O/P → true
boolean
false
boolean

2. Numbers

→ Unlike C, C++ & Java, JS doesn't believe in int, float, double etc.

→ Rather in JS, every numeric value belongs to a common data type called as number.

30, 10, 1.5 also a number
and 3.4 is also a number.

EX- `let a = 10;`
`console.log(a);`
`console.log(typeof a);`

`a = 6.5;`
`console.log(a);`
`console.log(typeof a);`

O/P → 10
number
6.5
number

- But there is problem with number data type in JS & the problem is that when our values crosses the limit $(2^{53} - 1)$ to $(2^{53}) - 1$ then the representation of the value changes to exponentiation.
- To overcome this problem we can force JS engine to represent the number of any size as BigInt.
- BigInt is a new data type introduced in 2020 and allowed us to represent numbers as large as beyond the limit $2^{53} - 1$ in their exact form.
- To make a number represent as BigInt, we suffix it with

→ underscore

Date _____
Page _____

the letter n.

Example-

```
let a = 12359723987312;
```

```
document.write(a);
```

```
document.write(<br>);
```

```
document.write(typeof a);
```

```
let b = 1234 97237698 n;
```

```
document.write(b);
```

```
document.write(<br>);
```

```
document.write(typeof b);
```

3. String

→ In JS any value enclosed in double " " and single ' ' is String.

→ "Bhopal" = String

'Bhopal' = String

"B" = String

'B' = String

EX → let city = "Bhopal";

```
document.write(city);
```

```
// Bhopal
```

```
document.write(<br>);
```

```
document.write(typeof city);
```

```
// String
```


var FirstName;

→ camel case notation

city = 'Bhopal'; // String
city = "B"; // String
city = 'B'; // String

→ If we want to use single quote and double quote as a part of string then we must enclose it in double quote and single quote.

EX → let mess = "lets' learn JS";
// error

let mess = "lets' learn JS";
// ok

let mess = "lets" learn JS;
// error

4. like C/C++ is also support back slash \ to represent single quote or double quote or a part of string.

4. Undefined and null

<Script>

var FirstName = "Sachin";

var LastName = 'Kapoor';

const age = 45;

console.log(FirstName);

console.log(LastName);

console.log(age);

→ It means a variable has been declared, but no value has been assigned to it.

→ In simple words we can say that, JavaScript automatically assigns undefined to any variable which is declared but not initialized.

```
var age; // declared but not initialized
console.log(age); // will show undefined
```

Null

→ It is a value that is assigned by programmer, to indicate that the variable currently has no value but will be initialized later.

→ In simple words we can say that, null represents intentional absence of the value.

→ In JS, undefined is a value as well as datatype both.

In JS, there are 5 situations where undefined is automatic-

usually used by JS.

- (a) whenever we declare a variable using var or let but we do not assign any value then JS assigns undefined.

Example-
`let a;
var b;
document.write(a);
document.write("
");
document.write(b);`

O/P → undefined
undefined

- (b) whenever we access an array beyond its range of indexes then JS gives undefined.

Example-
`let arr = [10, 20, 30];
document.write(arr[4]);`

O/P → undefined

- (c) whenever we access a non-existing property in an object then also JS gives undefined.

Example-
`let actor = {}`

name: "Manoj"
surname: "KaPoor";

document.write(actor.age);

O/P → undefined

(d) calling a Parameterized Fxn without argument

Example -

```
function show(a) {  
    document.write(a);  
}  
show();
```

O/P - undefined

(e) If a Fxn does not return anything the JS automatically returns undefined

Example - function show() {
 let a = 10;

document.write(show());

O/P → undefined

* NULL

1. NULL is also a value as well as datatype
2. In JS, whenever we have declared a variable but we don't know what value should be assigned to that variable then we can assign null and later on update that value.

EX → let currentTemp = null;

* Displaying value of a variable

1. In JS, to display a variable's value we can make use of + operator as shown below.

```
let name = "Sachin";  
document.write("my name is"  
               + name);
```

```
let age = 45;  
document.write("<br");  
document.write("my name is"  
               + age);
```

O/P → my name is Sachin
my age is 45

2. In JS + operator has dual

behaviour

(a) when used with numbers,
it performs addition.

(b) when used with string, it
performs concatenation.

EX → 1. "Good morning" = Good morning

2. "Good" + 10 = Good10

3. 10 + 20 + "Good" → 30 Good

4. 10 + "Good" + 20 → 10 Good 20

5. "Good" + 10 + 20 → Good 10 20

* Using template string or
template literals

Code -

→ In JS from ES6 onwards a
much simpler way of displayi-
ng variables along with a
message was introduced called
as template strings or template
literals.

→ It helps us avoid using '+' operator multiple times for concatenation.

→ To use template strings we take 2 steps:

(a) Enclose the entire message in back ticks.

(b) Use `${<var-name>}` whenever we want to display variable's value within the message.

Example-

```
let city = "Bhopal";  
let state = "MP";  
document.write(`capital of  
${state} is ${city}`);
```

* Dynamic nature of variables in JS

```
<script>
```

```
let name = "Sachin";  
document.write(`My name is  
${name}`);
```

```
document.write(`<br>name is  
of type ${typeof name}.  
${typeof name}<br>`);
```



```
name = 46;  
document.write('<br> And  
my age is ' + name);  
document.write('<br> now  
name is of type ' + (typeof  
name) + '<br>');  
</script>
```

O/P → My name is Sachin
name is of type string
And my age 46
Now name is of type
number

→ In JS a single var can have
diff types or values at
diff moments of time.